sluicebox

A learn-by-building ELT pipeline that mines job market intelligence from 130k+ LinkedIn postings.

» submit a suggestion · » report a bug · » contact

Forks 0 Stars 0

CI pull requests 0 open | last commit today | license Apache-2.0 | contributions welcome

Table of Content

1. About
   - Tech Stack
   - Features
2. Documentation
   - Setup
   - Development
3. Contributing
4. License

About

sluicebox is an end-to-end ELT pipeline built on the LinkedIn Job Postings 2023–2024 dataset — 130k+ real job listings with titles, descriptions, salaries, skills, and company metadata. The pipeline extracts raw postings, cleans and parses them into structured signals (salary ranges, skill flags, experience levels), loads them into a normalized analytics warehouse, and answers real questions about the data engineering job market using SQL.

> A sluice box is a gold mining tool that separates valuable material from raw rock. That's the pipeline.

:hammer_and_wrench: Tech Stack

➕ Development Tools

- ☑️ Python 3.11+
- ☑️ pytest + pytest-cov
- ☑️ GitHub Actions (CI)

➕ Data & Backend

- ☑️ pandas — chunked extraction and transformation
- ☑️ NumPy — vectorized operations
- ☑️ SQLAlchemy — database abstraction and connection management
- ☑️ SQLite — relational storage, swappable for PostgreSQL or SQL Server

➕ DevOps

- ✅ GitHub Actions — automated test runs on every push to `main`
- ✅ `.github/ISSUE_TEMPLATE` — structured bug, feature, chore, and deployment templates

🍄 Features

➕ Extract

- ✅ Class-based `JobExtractor` with chunked-read and full-load modes
- ✅ Separate `extract_companies()` loader for the companion companies CSV
- ✅ `profile()` utility — reports nulls, dtypes, unique counts, and samples per column before transforms run
- ✅ `detect_mixed_types()` — quantifies numeric vs string mix in chaotic columns like salary

➕ Transform

- ✅ `parse_salary()` — normalizes 6 salary formats (`$80K–$120K`, `95K`, `$75/hr`, plain float, null, free text) into `salary_low`, `salary_mid`, `salary_high` floats using regex named groups
- ✅ `strip_html()` / `clean_description()` — removes HTML tags from raw descriptions using stdlib `html.parser`, computes description length
- ✅ `extract_skills()` — dictionary-driven, whole-word regex detection of 18 tech skills; produces binary `skill_{name}` flag columns and a `skill_count` total
- ✅ `normalize_location()` — cleans location strings, flags remote postings
- ✅ `normalize_experience()` — maps raw experience level values to standard buckets via a `LEVEL_MAP` dictionary
- ✅ `validate_schema()` — circuit breaker that raises before a malformed DataFrame can reach the database
- ✅ Configurable `PIPELINE` list — add, remove, or reorder transforms without touching orchestration logic
- ✅ Row-count logging at every step for full pipeline observability

➕ Load

- ✅ Star schema: `dim_companies`, `dim_dates`, `fact_postings`
- ✅ `dim_dates` derived dynamically from posting months found in the data — no static seed file
- ✅ Dependency-ordered loading: dimensions before facts to satisfy foreign keys
- ✅ Idempotent full-refresh load — safe to re-run without duplicating data
- ✅ DDL version-controlled in `sql/schema.sql` alongside the code

➕ SQL Analytics

- ✅ Top 10 most demanded skills with % share of all postings
- ✅ Average, min, and max salary by experience level
- ✅ Salary premium per skill — avg salary with vs without each skill, and the delta
- ✅ Skill co-occurrence with Python — which skills appear most alongside it
- ✅ Monthly posting volume with running total and 3-month rolling average (window function)

- ✅ Remote vs on-site salary comparison by experience level

➕ Testing

- ✅ Unit tests for every UDF covering normal cases, edge cases, and null inputs
- ✅ `@pytest.mark.parametrize` on salary parsing — 6 input formats in one test function
- ✅ pytest fixtures for shared sample DataFrames
- ✅ 80%+ code coverage target enforced in CI

🌵 Documentation

🫙 Setup

Prerequisites: Python 3.11+, a Kaggle account with API credentials configured at `~/.kaggle/kaggle.json`.

```
# 1. Clone the repo
git clone https://github.com/KnowPlay/sluicebox.git
cd sluicebox

# 2. Create and activate a virtual environment
python -m venv .venv
source .venv/bin/activate        # Windows: .venv\Scripts\activate

# 3. Install dependencies
pip install -r requirements.txt

# 4. Download the source dataset
kaggle datasets download arshkon/linkedin-job-postings --path data/raw/ --
unzip
```

🍎 Development

Run the full pipeline:

```
python -m etl.pipeline
```

Initializes the database, extracts job postings and company data, applies all transforms, validates the output schema, and loads into the star schema warehouse. Every step is logged to stdout with timestamps and row counts.

Run the test suite:

```
pytest tests/ -v                 # all tests with verbose output
pytest tests/ --cov=etl          # with coverage report
```

Query the warehouse:

```
import sqlite3, pandas as pd

conn = sqlite3.connect("sluicebox.db")

# example: top 10 skills by posting frequency
df = pd.read_sql("""
    SELECT skill, SUM(mentions) AS total_postings,
           ROUND(100.0 * SUM(mentions) / (SELECT COUNT(*) FROM
fact_postings), 1) AS pct
    FROM (
        SELECT 'python' AS skill, SUM(skill_python) AS mentions FROM
fact_postings UNION ALL
        SELECT 'sql',              SUM(skill_sql)               FROM
fact_postings UNION ALL
        SELECT 'spark',            SUM(skill_spark)             FROM
fact_postings UNION ALL
        SELECT 'aws',              SUM(skill_aws)               FROM
fact_postings
    )
    GROUP BY skill ORDER BY total_postings DESC
""", conn)
```

Or open `sql/analytics.sql` directly in any SQLite client pointed at `sluicebox.db`.

Project structure:

```
sluicebox/
├── .github/
│   ├── ISSUE_TEMPLATE/     # bug, feature, chore, deployment,
documentation templates
│   └── workflows/
│       └── ci.yml          # runs pytest on every push to main
├── data/
│   ├── raw/                # source CSVs — gitignored, download via
Kaggle CLI
│   └── processed/          # intermediate outputs
├── etl/
│   ├── extract.py          # JobExtractor — chunked, full-load, and
company modes
│   ├── transform.py        # UDF library, PIPELINE list, schema validator
│   ├── load.py             # DB init, dimension + fact loaders, load_all
orchestrator
│   ├── pipeline.py         # single entry point: extract → transform →
load
│   └── utils.py            # profile(), detect_mixed_types(), shared
helpers
├── sql/
```

```
│   ├── schema.sql          # DDL — dim_companies, dim_dates,
fact_postings
│   └── analytics.sql       # 6 analytical queries
├── tests/
│   ├── test_transform.py   # UDF unit tests — fixtures, parametrize, edge
cases
│   └── test_load.py        # load function tests with mocked DB
├── config.py               # paths, DB_URL, get_logger() factory
├── requirements.txt
└── README.md
```

Key design decisions:

- **Functional transform chain** — each UDF is a pure function (DataFrame in, DataFrame out). Pure functions are independently testable, composable, and produce no side effects. The `PIPELINE` list means steps can be reordered without touching orchestration logic.
- **Idempotent loading** — the load step uses truncate-and-replace so re-running the pipeline never produces duplicate rows. Critical for pipelines that get re-triggered on failure.
- **Schema validation as a gate** — `validate_schema()` runs between transform and load, acting as a circuit breaker that fails fast before bad data reaches the database.
- **Denormalized skill flags** — skill presence is stored as individual integer columns on `fact_postings` rather than a separate skills table. This is a deliberate tradeoff: it makes co-occurrence and aggregation queries significantly faster at the cost of a wider table.
- **Config-driven paths** — all file paths and the DB URL live in `config.py`, never hardcoded. Swap SQLite for PostgreSQL by changing one line.

🌾 Contributing

Contributions, questions, and suggestions are welcome — especially from others learning data engineering.

> 1. Fork the Project
> 2. Create your Branch (`git checkout -b my-branch`)
> 3. Commit your Changes (`git commit -m 'add my contribution'`)
> 4. Push to the Branch (`git push --set-upstream origin my-branch`)
> 5. Open a Pull Request

📝 License

This project is licensed under the Apache License 2.0.